

 [Volver a 'Recuperatorio'](#)

Comenzado el	miércoles, 21 de julio de 2021, 18:22
Estado	Finalizado
Finalizado en	miércoles, 21 de julio de 2021, 21:19
Tiempo empleado	2 horas 57 minutos
Puntos	48,00/80,00
Calificación	6,00 de 10,00 (60%)

Pregunta **1**

Finalizado

Puntúa 6,00 sobre 10,00

Definir las funciones **duracion** :: **Comp** -> **Int**, que describe la duración total de la composición dada, y **alargar** :: **Comp** -> **Int** -> **Comp**, que dada una composición, describe la composición que resulta de alargar cada nota o silencio por el factor dado.

ATENCIÓN: recordar que ***NO se puede retroceder*** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
duracion :: Comp -> Int
duracion (Silencio d) = d
duracion (Pulso nota d) = d
duracion (Arpeggio c1 c2) = duracion c1 + duracion c2
duracion (Acorde c1 c2) = if (duracion c1 > duracion c2)
                                then duracion c1
                                else duracion c2

alargar :: Comp -> Int -> Comp
alargar (Silencio d) n = Silencio (d+n)
alargar (Pulso nota d) n = Pulso nota (d+n)
alargar (Arpeggio c1 c2) n = Arpeggio (alargar c1 n) (alargar c2 n)
alargar (Acorde c1 c2) n = Acorde (alargar c1 n) (alargar c2 n)
```

Comentario:

NOTA: R+

OBSERVACIONES:

- * La función alargar debería multiplicar, porque si no se defasan los acordes de diferente duración...
- * En la función duración podrías haber usado la función max.

Pregunta **2**

Finalizado

Puntúa 6,00
sobre 10,00

Definir la función `sintetizar :: Comp -> Midi`, que describe el **Midi** equivalente a la composición dada.

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
sintetizar :: Comp -> Midi
duracion (Silencio d) = []
duracion (Pulso nota d) = [[nota]]
duracion (Arpeggio c1 c2) = (sintetizar c1) ++ (sintetizar c2)
duracion (Acorde c1 c2) = concat (sintetizar c1) (sintetizar c2) --concat función utiliza
```

Comentario:

NOTA: R-

OBSERVACIONES:

- * La longitud del Midi debería ser la misma que la duración que la composición, porque si no al ponerla en paralelo, se defasa. Por eso los casos Silencio y Pulso están mal.
- * La función concat :: [[a]] -> [a], y vos la estás usando con dos listas, con error de tipos. Si la usases sobre dos listas con tipo correcto, sería equivalente a (++), pero en ese caso no expresaría el paralelismo de Acorde.

Pregunta **3**

Finalizado

Puntúa 6,00
sobre 10,00

Demostrar que para todo `k :: Int`, para todo `c :: Comp`. `k >= 0` entonces `duracion (alargar c k) = k * duracion c`.

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
Demostrar que para todo k :: Int, para todo c :: Comp.
k >= 0 entonces duracion (alargar k c) = k * duracion c.

Hi) k >= 0

--Para todo c
--Para todo k
¿duracion (alargar c k) = k * duracion c ?

sea c un Comp bien definido, demuestro con inducción sobre la estructura de k.
```

Comentario:

NOTA: R+, B, R+

OBSERVACIONES:

- * En el planteo se confunde la Hipótesis con una HI, e inmediatamente se la vuelve a confundir al eliminarla de todos los casos (pero conservar el para todo k).
- * En la justificación para realizar el planteo, se habla de hacer recursión sobre k en lugar de sobre c...
- * La propiedad distributiva del caso inductivo 2 solamente es válida si `k >= 0`, cosa que no se menciona ni se justifica.

Pregunta **4**

Finalizado

Puntúa 6,00
sobre 10,00

Indicar el tipo y dar una implementación de una función que exprese la recursión primitiva sobre las composiciones.

ATENCIÓN: recordar que ***NO se puede retroceder*** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
recrT:: (Duracion -> b) -> (Nota -> Duracion -> b) -> (Comp -> Comp -> b -> b -> b) -> (C
recrT  fd fn fc1 fc2 (Silencio d) = fd d
recrT  fd fn fc1 fc2 (Pulso n d) = fn n d
recrT  fd fn fc1 fc2 (Arpegio c1 c2) = fc1 c1 c2 (recrT fd fn fc1 fc2 c1) (recrT fd fn fc
recrT  fd fn fc1 fc2 (Acorde c1 c2) =  fc2 c1 c2 (recrT fd fn fc1 fc2 c1) (recrT fd fn fc
```

Comentario:

NOTA: B

OBSERVACIONES:

* Bien

Pregunta **5**

Finalizado

Puntúa 6,00
sobre 10,00

Indicar el tipo y dar una implementación de una función que exprese la recursión estructural sobre las composiciones sin utilizar recursión explícita.

ATENCIÓN: recordar que ***NO se puede retroceder*** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
foldC:: (Duracion -> b) -> (Nota -> Duracion -> b) -> (b -> b -> b) -> (b -> b -> b) -> C
foldC fd fn fc1 fc2 = recrT fd fn (\_ _ r1 r2 ->fc1 r1 r2) (\_ _ r1 r2 ->fc2 r1 r2)
```

Comentario:

NOTA: B

OBSERVACIONES:

* Bien

Pregunta **6**
Finalizado
Puntúa 6,00
sobre 10,00

Definir, *expresadas como recursión estructural implícita*, las funciones del ejercicio 1 (**duracion** :: **Comp** -> **Int**, que describe la duración total de la composición dada, y **alargar** :: **Comp** -> **Int** -> **Int**, que dada una composición, describe la composición que resulta de alargar cada nota o silencio por el factor dado.)

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
duracion :: Comp -> Int
duracion = foldTC (\d -> d) (\n d -> d) (\r1 r2 -> r1 + r2) (\r1 r2 -> if (r1 > r2)

alargar :: Comp -> Int -> Int
alargar = foldTC (\d n -> Silencio (d*n)) (\n d n2 -> Pulso n (d*n2)) (\r1 r2 n -> Arpe
```

Comentario:

NOTA: B

OBSERVACIONES:

* Bien

Pregunta **7**
Finalizado
Puntúa 6,00
sobre 10,00

Definir, *expresada como recursión estructural implícita*, la función del ejercicio 2 (**sintetizar** :: **Comp** -> **Midi**, que describe el **Midi** equivalente a la composición dada).

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
--Vuelvo a definir sintetizar del ejercicio 2 ya que la hice mal.

sintetizar :: Comp -> Midi
sintetizar (Silencio d) = []
sintetizar (Pulso nota d) = map (nota:) (sampler d)
sintetizar (Arpeggio c1 c2) = (sintetizar c1) ++ (sintetizar c2)
sintetizar (Acorde c1 c2) = concat (sintetizar c1) (sintetizar c2)

sintetizar :: Comp -> Midi
sintetizar = foldTC (\d -> []) (\n d -> map (n:) (sampler d)) (\r1 r2 -> r1 ++ r2) (\r1
```

Comentario:

NOTA: R-

OBSERVACIONES:

* La nueva versión de sintetizar con recursión explícita sigue siendo incorrecta (el caso Silencio y el caso Acorde no cambiaron, y en el caso Pulso se usa una funcion inexistente, sampler)

Pregunta **8**

Finalizado

Puntúa 6,00
sobre 10,00

Definir, *sin utilizar recursión estructural explícita*, la función `unir :: [Comp] -> [Comp] -> Comp`, que dadas dos listas de composiciones de igual longitud, describe una composición donde las partes de la misma son las composiciones de cada posición de las listas dadas sonando en simultáneo. Así, las dos primeras composiciones suenan juntas, y al terminar suenan juntas las dos segundas, luego las dos terceras juntas, etc.

ATENCIÓN: recordar que ***NO se puede retroceder*** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
unir :: [Comp] -> [Comp] -> Comp
unir cs cs' = foldListas g d

      where g [] (c:cs) = Acorde (Acorde (Silencio d) c)
            where d (c1:cs1) (c2:cs2) x r = Acorde (Acorde c1 c2) (r
```

Comentario:

NOTA: R

OBSERVACIONES:

- * La función foldListas no está definida, y por lo tanto no se puede evaluar si las definiciones de g y d son correctas.
- * La función g parecería combinar una lista vacía con una no vacía, lo que no puede suceder por precondition (listas de igual longitud); también descarta cs. Además, le falta el caso [] para la 2da lista.
- * La función d parece bien encaminada, pero no queda claro quién es r...

◀ [Recuperatorio - Dominio](#)

Ir a...

[Clase teórica por streaming: Modelo funcional](#) ▶